

# Trabajo Práctico N.º 5: Optimización y Procesamiento de Datos en MongoDB

Nombre: Facundo Arrieta

## Actividad 1: Optimización mediante Índices

### 1) Optimización de consultas frecuentes en una empresa de logística

Una empresa de logística gestiona miles de envíos. Las consultas más frecuentes buscan paquetes por su `numero_seguimiento`, el cual es único, y también filtran por estado del envío y `ciudad_destino` de manera simultánea.

#### a) Índice simple

El campo que requiere un índice simple es `numero_seguimiento`, ya que es utilizado frecuentemente para buscar paquetes específicos y, además, su valor es único para cada envío.

El comando propuesto para crear el índice simple es:

```
db.envios.createIndex({ numero_seguimiento: 1 }, { unique: true })
```

Este índice permite que MongoDB localice rápidamente un envío a partir de su número de seguimiento, evitando recorrer toda la colección.

#### b) Índice compuesto

Los campos que requieren un índice compuesto son `estado` y `ciudad_destino`, ya que las consultas frecuentes filtran por ambos campos al mismo tiempo.

El comando propuesto para crear el índice compuesto es:

```
db.envios.createIndex({ estado: 1, ciudad_destino: 1 })
```

Este índice optimiza las búsquedas que combinan el estado del envío con la ciudad de destino, permitiendo que MongoDB acceda a los documentos de manera más eficiente.

#### c) Búsqueda en una colección grande sin índices

Cuando se realiza una búsqueda en una colección de gran volumen que no posee índices, MongoDB debe recorrer todos los documentos de la colección uno por uno para encontrar aquellos que cumplen con la condición solicitada. Este proceso se conoce como escaneo completo de colección o COLLSCAN.

El problema de este comportamiento es que, a medida que aumenta la cantidad de documentos, la consulta puede volverse más lenta y consumir más recursos. Por eso, los índices son importantes para mejorar el rendimiento, ya que permiten acceder a los datos de forma más directa y eficiente.

## 2. Implementación práctica en mongosh

### a) Creación de la base de datos

```
test> use logistica_db
switched to db logistica_db
logistica_db>
```

use logistica\_db

### b) Creación de la colección

```
logistica_db> db.createCollection("envios")
{ ok: 1 }
logistica_db>
```

db.createCollection("envios")

### c) Inserción de documentos

```
logistica_db> db.envios.insertMany([
  {
    numero_seguimiento: "ENV001",
    estado: "Entregado",
    ciudad_destino: "Medellin",
    peso: 12
  },
  {
    numero_seguimiento: "ENV002",
    estado: "En tránsito",
    ciudad_destino: "San Juan",
    peso: 25
  },
  {
    numero_seguimiento: "ENV003",
    estado: "Pendiente",
    ciudad_destino: "Córdoba",
    peso: 8
  },
  {
    numero_seguimiento: "ENV004",
    estado: "Entregado",
    ciudad_destino: "Medellin",
    peso: 30
  },
  {
    numero_seguimiento: "ENV005",
    estado: "En tránsito",
    ciudad_destino: "Buenos Aires",
    peso: 18
  }
])
logistica_db> db.envios.insertMany([
  {
    numero_seguimiento: "ENV001",
    estado: "Entregado",
    ciudad_destino: "Medellin",
    peso: 12
  },
  {
    numero_seguimiento: "ENV002",
    estado: "En tránsito",
    ciudad_destino: "San Juan",
    peso: 25
  },
  {
    numero_seguimiento: "ENV003",
    estado: "Pendiente",
    ciudad_destino: "Córdoba",
    peso: 8
  }
])
acknowledged: true,
insertedIds: {
  '0': ObjectId('6a1b1bf9349746be693682d1'),
  '1': ObjectId('6a1b1bf9349746be693682d2'),
  '2': ObjectId('6a1b1bf9349746be693682d3'),
  '3': ObjectId('6a1b1bf9349746be693682d4'),
  '4': ObjectId('6a1b1bf9349746be693682d5'),
  '5': ObjectId('6a1b1bf9349746be693682d6'),
  '6': ObjectId('6a1b1bf9349746be693682d7'),
  '7': ObjectId('6a1b1bf9349746be693682d8'),
  '8': ObjectId('6a1b1bf9349746be693682d9'),
  '9': ObjectId('6a1b1bf9349746be693682da')
```

db.envios.insertMany([

{

```
numero_seguimiento: "ENV001",
estado: "Entregado",
ciudad_destino: "Mendoza",
peso: 12
},
{
numero_seguimiento: "ENV002",
estado: "En tránsito",
ciudad_destino: "San Juan",
peso: 25
},
{
numero_seguimiento: "ENV003",
estado: "Pendiente",
ciudad_destino: "Córdoba",
peso: 8
},
{
numero_seguimiento: "ENV004",
estado: "Entregado",
ciudad_destino: "Mendoza",
peso: 30
},
{
numero_seguimiento: "ENV005",
estado: "En tránsito",
ciudad_destino: "Buenos Aires",
peso: 18
```

```
},  
{  
  numero_seguimiento: "ENV006",  
  estado: "Entregado",  
  ciudad_destino: "San Juan",  
  peso: 22  
},  
{  
  numero_seguimiento: "ENV007",  
  estado: "Pendiente",  
  ciudad_destino: "Mendoza",  
  peso: 5  
},  
{  
  numero_seguimiento: "ENV008",  
  estado: "Entregado",  
  ciudad_destino: "Córdoba",  
  peso: 15  
},  
{  
  numero_seguimiento: "ENV009",  
  estado: "Cancelado",  
  ciudad_destino: "Buenos Aires",  
  peso: 10  
},  
{  
  numero_seguimiento: "ENV010",  
  estado: "En tránsito",
```

```
    ciudad_destino: "Mendoza",  
    peso: 28  
  }  
  ]  
}
```

#### d) Creación de índices

índice simple sobre numero\_seguimiento

```
logistica_db> db.envios.createIndex(  
|   { numero_seguimiento: 1 },  
|   { unique: true }  
| )  
numero_seguimiento_1  
logistica_db>
```

```
db.envios.createIndex(  
  { numero_seguimiento: 1 },  
  { unique: true }  
)
```

Índice compuesto sobre estado y ciudad\_destino

```
logistica_db> db.envios.createIndex(  
|   { estado: 1, ciudad_destino: 1 }  
| )  
estado_1_ciudad_destino_1  
logistica_db>
```

```
db.envios.createIndex(  
  { estado: 1, ciudad_destino: 1 }  
)
```

## Actividad 2: Diagnóstico y Evaluación de Rendimiento

### 1) Consulta con bajo rendimiento

El equipo de desarrollo nota que una consulta específica está demorando demasiado tiempo en ejecutarse.

a) Comando para visualizar COLLSCAN o IXSCAN

El comando que permite analizar cómo se ejecuta una consulta en MongoDB es `explain()`, utilizando el modo "executionStats".

Ejemplo:

```
db.envios.find({ numero_seguimiento: "ENV001" }).explain("executionStats")
```

Este comando permite visualizar el plan de ejecución de la consulta y determinar si MongoDB realizó un escaneo completo de la colección, identificado como COLLSCAN, o si utilizó un índice, identificado como IXSCAN.

b) Significado de `executionTimeMillis` y `totalDocsExamined`

El campo `executionTimeMillis` indica el tiempo total que tardó MongoDB en ejecutar la consulta, expresado en milisegundos. Este valor permite evaluar si una consulta está demorando demasiado.

El campo `totalDocsExamined` indica la cantidad total de documentos que MongoDB tuvo que revisar para resolver la consulta. Si este número es muy alto en comparación con la cantidad de documentos devueltos, puede ser señal de que la consulta no está optimizada o de que no se está utilizando un índice adecuado.

c) Solución ante uso alto de memoria por documentos grandes

Si se observa un uso alto de memoria debido a documentos muy grandes o demasiado anidados, la solución recomendada es simplificar la estructura de los documentos o utilizar subcolecciones. Esto permite reducir el tamaño de los documentos principales y mejorar el rendimiento general de la base de datos.

2) Implementación práctica en mongosh:

a)

```
logistica_db> db.envios.find({ numero_seguimiento: "ENV001" }).explain("executionStats")
```

```
{
  db.envios.find({ numero_seguimiento: "ENV001"
}).explain("executionStats")
```

- b) Al analizar el resultado de `explain("executionStats")`, se observa que el plan ganador utiliza el stage `EXPRESS_IXSCAN`. Esto indica que MongoDB utilizó un índice para resolver la consulta. Además, el resultado muestra el índice utilizado:

`indexName: "numero_seguimiento_1"`

Por lo tanto, se verifica que la consulta por `numero_seguimiento` utilizó correctamente el índice simple creado en la Actividad 1.

```
logistica_db.envios.find( { numero_seguimiento: 'ENV001' } )
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'logistica_db.envios',
    parsedQuery: { numero_seguimiento: { '$eq': 'ENV001' } },
    indexFilterSet: false,
    queryHash: 'F0184EF5',
    planCacheShapeHash: 'F0184EF5',
    planCacheKey: '957F69BE',
    optimizationTimeMillis: 0,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    prunedSimilarIndexes: false,
    winningPlan: {
      isCached: false,
      stage: 'EXPRESS_IXSCAN',
      keyPattern: '{ numero_seguimiento: 1 }',
      indexName: 'numero_seguimiento_1'
    },
    rejectedPlans: []
  },
  executionStats: {
    executionSuccess: true,
    nReturned: 1,
    executionTimeMillis: 0,
    totalKeysExamined: 1,
    totalDocsExamined: 1,
    executionStages: {
      isCached: false,
      stage: 'EXPRESS_IXSCAN',
      keyPattern: '{ numero_seguimiento: 1 }',
      indexName: 'numero_seguimiento_1',
      keysExamined: 1,
      docsExamined: 1,
      nReturned: 1
    }
  }
}
```

c) `db.envios.stats()`

El valor obtenido en el campo `totalIndexSize` fue:

`totalIndexSize: 61440`

Esto indica que el tamaño total ocupado por los índices de la colección `envios` es de 61.440 bytes.

### Actividad 3: Procesamiento Avanzado con Agregaciones

#### 1) Reportes financieros y operativos de los envíos realizados

##### a) Función del Pipeline de Agregación

El Pipeline de Agregación en MongoDB permite procesar, transformar y resumir datos dentro de una colección mediante una serie de etapas.

Funciona como una “tubería” porque los documentos van pasando por distintas etapas de procesamiento. La salida de una etapa se convierte en la entrada de la siguiente, hasta obtener el resultado final.

Este mecanismo permite generar reportes, calcular estadísticas, filtrar información, agrupar datos y transformar documentos de manera eficiente dentro de MongoDB.

##### b) Etapas `$match`, `$group` y `$project`

La etapa `$match` se utiliza para filtrar documentos según una condición determinada. Funciona de manera similar a un `WHERE` en SQL, ya que permite seleccionar solo los documentos que cumplen con ciertos criterios.

La etapa `$group` se utiliza para agrupar documentos según un campo específico y aplicar operaciones de agregación, como contar documentos, sumar valores o calcular promedios.

La etapa `$project` se utiliza para seleccionar, ocultar o transformar campos en el resultado final. Permite definir qué campos se mostrarán y también crear nuevos campos a partir de otros existentes.

##### c) Operador para unir la colección `envios` con una colección de clientes

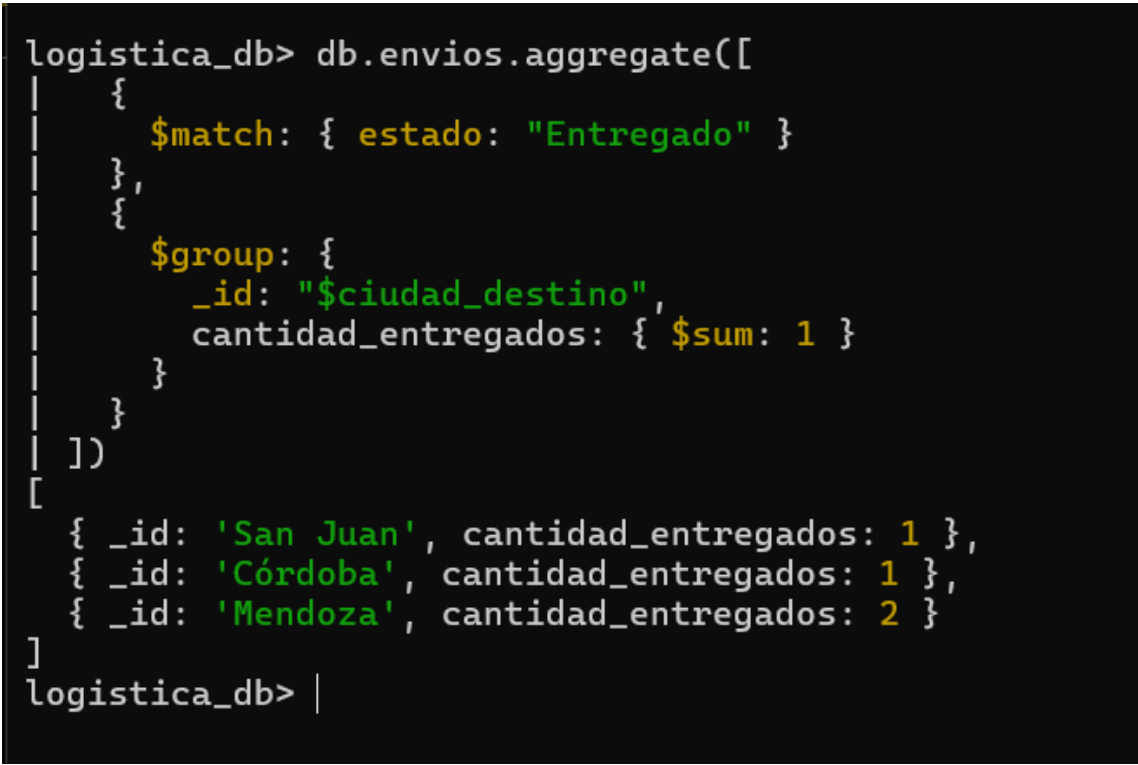
Para unir la colección `envios` con una colección de clientes, se utilizaría el operador de agregación `$lookup`.

Este operador permite realizar una unión entre colecciones, similar a un JOIN en bases de datos relacionales. De esta manera, se puede combinar información de los envíos con los datos correspondientes de los clientes.

## 2) Aplicación en Mongosh

### a) Filtrar envíos "Entregado" y agrupar por ciudad

```
db.envios.aggregate([
  {
    $match: { estado: "Entregado" }
  },
  {
    $group: {
      _id: "$ciudad_destino",
      cantidad_entregados: { $sum: 1 }
    }
  }
])
```



```
logistica_db> db.envios.aggregate([
|   {
|     $match: { estado: "Entregado" }
|   },
|   {
|     $group: {
|       _id: "$ciudad_destino",
|       cantidad_entregados: { $sum: 1 }
|     }
|   }
| ])
|
| [
|   { _id: 'San Juan', cantidad_entregados: 1 },
|   { _id: 'Córdoba', cantidad_entregados: 1 },
|   { _id: 'Mendoza', cantidad_entregados: 2 }
| ]
logistica_db> |
```

### b) Calcular el peso total de todos los envíos

```
db.envios.aggregate([
  {
    $group: {
```

```

    _id: null,
    peso_total: { $sum: "$peso" }
  }
},
{
  $project: {
    _id: 0,
    peso_total: 1
  }
}
])

```

```

logistica_db> db.envios.aggregate([
  {
    $group: {
      _id: null,
      peso_total: { $sum: "$peso" }
    }
  },
  {
    $project: {
      _id: 0,
      peso_total: 1
    }
  }
])
[ { peso_total: 173 } ]
logistica_db>

```

c) Mostrar numero\_seguimiento en mayúsculas y el estado

```

logistica_db> db.envios.aggregate([
  {
    $project: {
      _id: 0,
      numero_seguimiento_mayuscula: { $toUpper: "$numero_seguimiento" },
      estado: 1
    }
  }
])
[
  { estado: 'Entregado', numero_seguimiento_mayuscula: 'ENV001' },
  { estado: 'En tránsito', numero_seguimiento_mayuscula: 'ENV002' },
  { estado: 'Pendiente', numero_seguimiento_mayuscula: 'ENV003' },
  { estado: 'Entregado', numero_seguimiento_mayuscula: 'ENV004' },
  { estado: 'En tránsito', numero_seguimiento_mayuscula: 'ENV005' },
  { estado: 'Entregado', numero_seguimiento_mayuscula: 'ENV006' },
  { estado: 'Pendiente', numero_seguimiento_mayuscula: 'ENV007' },
  { estado: 'Entregado', numero_seguimiento_mayuscula: 'ENV008' },
  { estado: 'Cancelado', numero_seguimiento_mayuscula: 'ENV009' },
  { estado: 'En tránsito', numero_seguimiento_mayuscula: 'ENV010' }
]
logistica_db>

```

d) Crear el campo tipo\_carga con \$cond

```

db.envios.aggregate([
  {
    $addFields: {
      tipo_carga: {
        $cond: {
          if: { $gt: ["$peso", 20] },
          then: "Pesada",
          else: "Ligera"
        }
      }
    }
  },
  {
    $project: {
      _id: 0,
      numero_seguimiento: 1,
      peso: 1,
      tipo_carga: 1
    }
  }
])

```

)

```
{ estado: 'En tránsito', numero_seguimiento_mayuscula: 'ENV010' }
]
logistica_db> db.envios.aggregate([
  {
    $addFields: {
      tipo_carga: {
        $cond: {
          if: { $gt: ["$peso", 20] },
          then: "Pesada",
          else: "Ligera"
        }
      }
    }
  },
  {
    $project: {
      _id: 0,
      numero_seguimiento: 1,
      peso: 1,
      tipo_carga: 1
    }
  }
])
[
  { numero_seguimiento: 'ENV001', peso: 12, tipo_carga: 'Ligera' },
  { numero_seguimiento: 'ENV002', peso: 25, tipo_carga: 'Pesada' },
  { numero_seguimiento: 'ENV003', peso: 8, tipo_carga: 'Ligera' },
  { numero_seguimiento: 'ENV004', peso: 30, tipo_carga: 'Pesada' },
  { numero_seguimiento: 'ENV005', peso: 18, tipo_carga: 'Ligera' },
  { numero_seguimiento: 'ENV006', peso: 22, tipo_carga: 'Pesada' },
  { numero_seguimiento: 'ENV007', peso: 5, tipo_carga: 'Ligera' },
  { numero_seguimiento: 'ENV008', peso: 15, tipo_carga: 'Ligera' },
  { numero_seguimiento: 'ENV009', peso: 10, tipo_carga: 'Ligera' },
  { numero_seguimiento: 'ENV010', peso: 28, tipo_carga: 'Pesada' }
]
logistica_db>
```

**Conclusión:** A través del presente trabajo práctico se aplicaron técnicas de optimización y procesamiento de datos en MongoDB. En primer lugar, se crearon índices simples y compuestos para mejorar el rendimiento de consultas frecuentes sobre la colección envios. Luego, se utilizaron herramientas de diagnóstico como explain("executionStats") y stats() para analizar el comportamiento de las consultas, verificar el uso de índices y obtener información sobre el tamaño total ocupado por los mismos.

Finalmente, se implementaron pipelines de agregación para generar reportes operativos, calcular totales, transformar campos y clasificar documentos mediante condiciones. Estas prácticas permiten comprender cómo MongoDB puede ser utilizado no solo para almacenar datos, sino también para optimizar búsquedas, analizar información y generar resultados útiles directamente desde la base de datos.

